

Movie Industry Data Warehouse

Financial & Critical Analysis Pipeline

End-to-end data warehousing pipeline analyzing the global movie industry's financial and critical performance, built with Python and PostgreSQL.

Student: Alfredo Segala

Project Goal & Research Questions

Build a reproducible enterprise Data Warehouse to analyze the global movie industry's financial performance and critical success, integrating heterogeneous datasets to support multidimensional OLAP analytics.

Historical & Geographic Trends

How has movie profitability evolved across different countries and decades?

Studio Dominance

Which production companies lead the market in total revenue, and how do they perform in specific blockbuster genres like Action?



Talent Reliability

Which actors and directors guarantee the best balance between box office net profit and critical acclaim?

Seasonal & Genre Demand

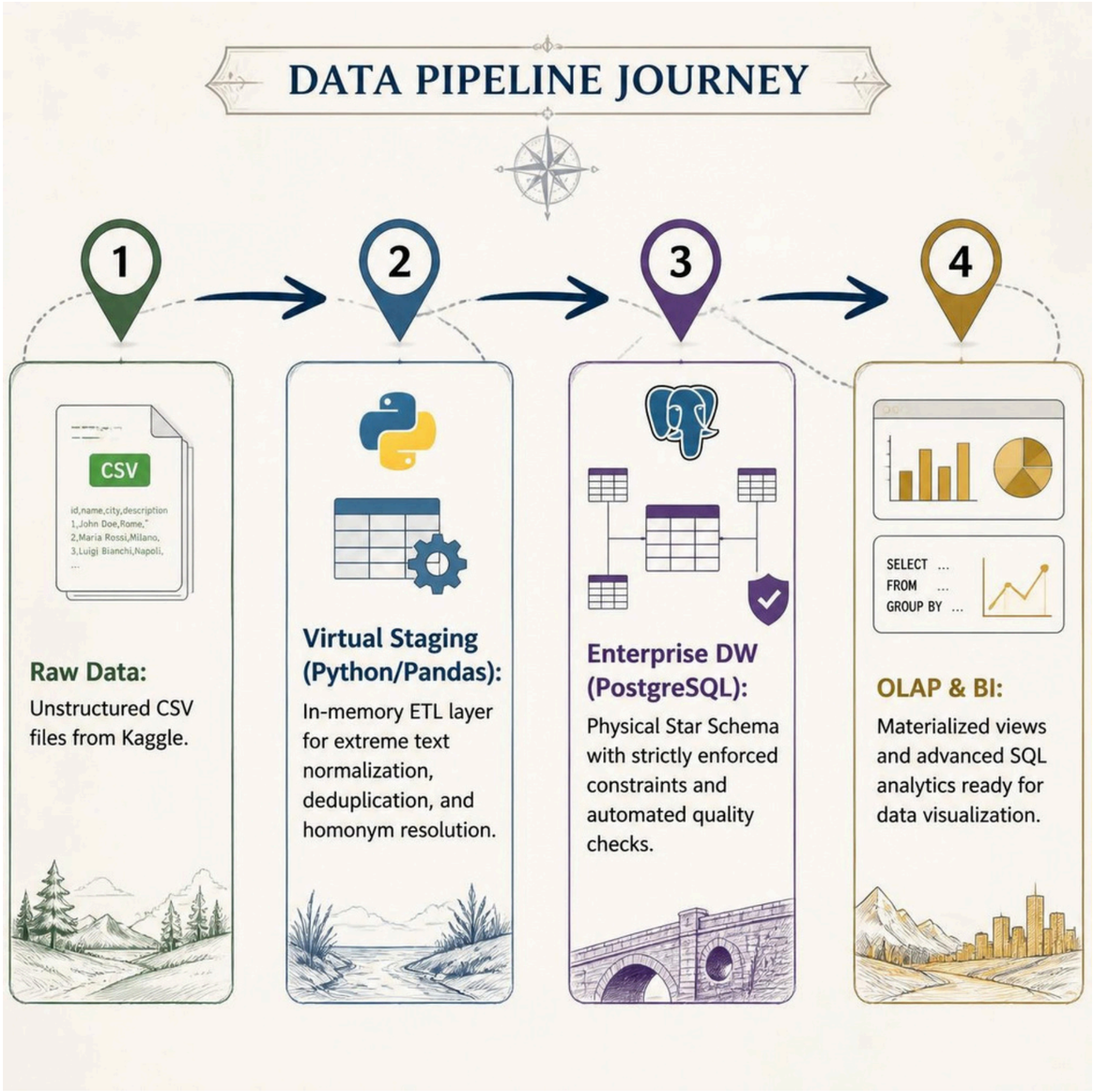
How do movie revenues fluctuate across different months, quarters, and primary genres?

Data Sources & Pipeline Architecture

Source	Content	Purpose
TMDB Dataset	Budgets, revenues, release dates	Core financial metrics
IMDb Dataset	Scores, genres, cast & directors	Critical reception & talent dimensions

The Main Challenge

No shared Unique Identifier (ID) between the two datasets. This required an advanced Virtual Staging phase using Python to normalize text and cross-merge records dynamically.



ETL Operations & Virtual Staging

Raw Sources Challenges

- Heterogeneous Kaggle CSVs with massive text inconsistencies.
- No shared unique identifiers (IDs) between TMDB and IMDb.
- Presence of homonyms (e.g., King Kong 1976 vs King Kong 2005).

The Virtual Staging Approach (Pandas)

To avoid overloading the database with raw, messy data, I implemented a Two-Layer Architecture using Python as an in-memory Virtual Staging area:

- Extreme Normalization: Stripping punctuation, spaces, and leading articles from titles to force absolute text matches.
- Data Deduplication: Dropping invalid records and generating unique deterministic titles for the Golden Record.

ETL Implementation & Auto-Detect Flow

4-Way Cross Merge

Merging datasets using 4 different title combinations (Original vs US titles) with a +/- 1 year release tolerance to resolve temporal discrepancies.

Dynamic Auto-Detect

A custom Python algorithm automatically scans the PostgreSQL dim_movie table to dynamically find the perfect alignment key with the TMDB dataframe.

PostgreSQL Injection

Loading the certified "Golden Record" (4,187 movies and 3,592 people) into the Star Schema using SQLAlchemy and generating automatic surrogate keys (UUIDs).

Conceptual Design: Dimensional Fact Model

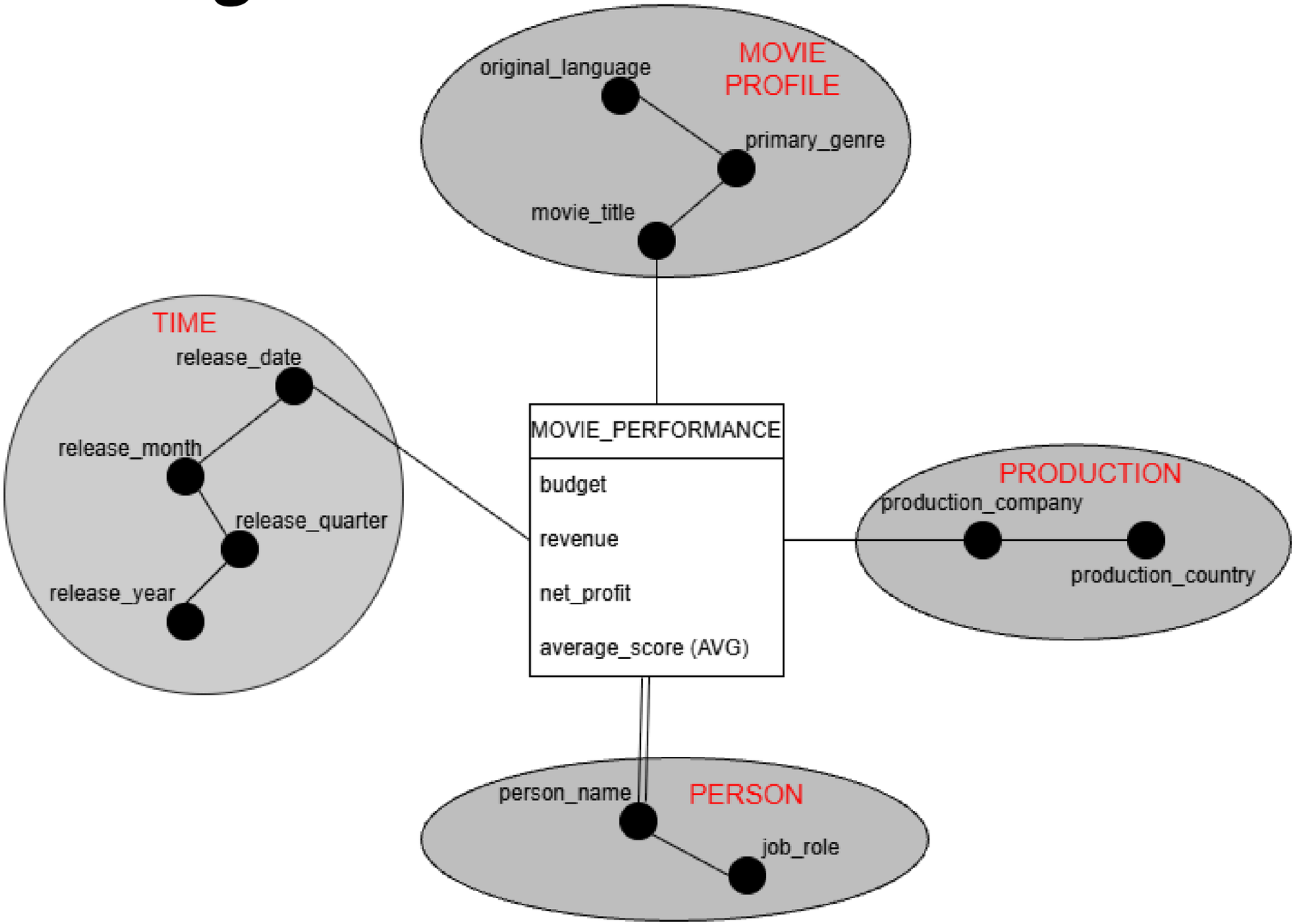
Conceptual Model Overview

The Dimensional Fact Model (DFM) defines the business logic independently from the relational database structure.

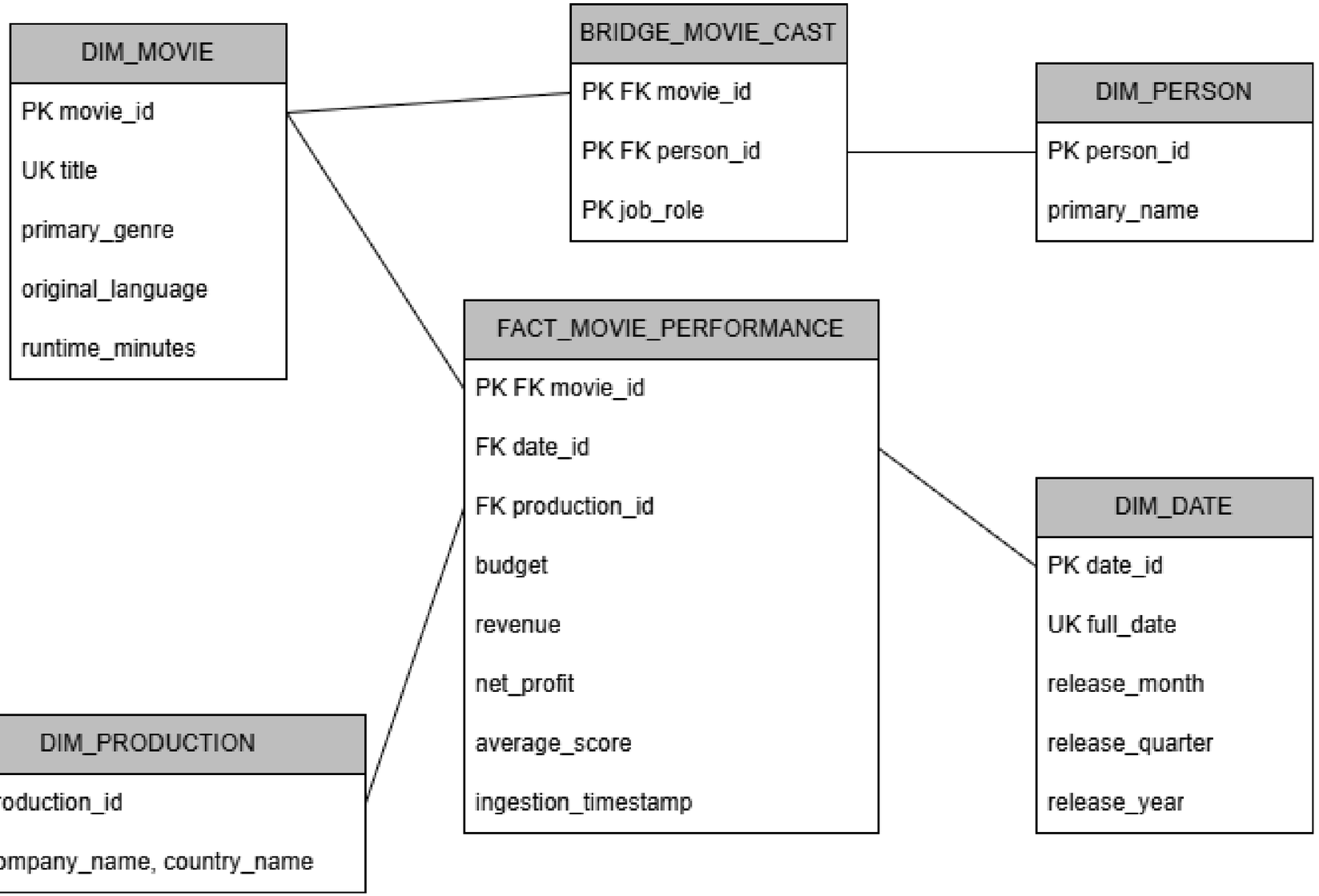
Central Fact: MOVIE_PERFORMANCE (Analytical grain: one movie release).

Analysis Dimensions: TIME, PRODUCTION, MOVIE PROFILE, and PERSON.

The Multiple Arc (N:M): The PERSON dimension is conceptually linked via a many-to-many relationship (multiple arc) to accurately represent the multiple actors and directors involved in a single movie event.



Physical Implementation: Star Schema



Translating DFM into PostgreSQL

Pure Star Architecture: A strictly denormalized structure around the central fact table to maximize OLAP query performance.

The Bridge Table Solution: To physically resolve the multiple arc from the DFM without causing silent data duplication (fan-out), I implemented the BRIDGE_MOVIE_CAST table (PK FK movie_id, PK FK person_id, PK job_role).

Referential Integrity: Enforced via primary and foreign keys, ensuring zero orphan records across all analytical dimensions.

Fact Table & Core Metrics

FACT_MOVIE_PERFORMANCE

Analytical Grain: One certified movie.

Core Measures:

- budget (Numeric, USD)
- revenue (Numeric, USD)
- net_profit (Calculated mathematically: revenue - budget)
- average_score (Critical reception, 0-100 scale)

Auditability: Includes an ingestion_timestamp for pipeline tracking.

Design Choice: Maintaining a single, granular fact table at the movie level guarantees fast ROLL-UP and DRILL-DOWN operations without the risk of overcounting financial data.

OLAP Query Design

OLAP Analytics & Query Design

Our OLAP queries extract analytical insights from the dimensional model using advanced SQL operations:

Slice, Dice & Roll-up

Aggregating financial data by country, decade, and primary genre to evaluate historical profitability.

Window Functions (ROW_NUMBER)

Extracting the "Golden Decade" for each production country without generating massive subqueries.

Drill-Across & UNION ALL

Combining the elite Top 10 Actors and Top 10 Directors into a single "Hall of Fame" ranking based on net profit and IMDb score.

LEFT JOIN Aggregations

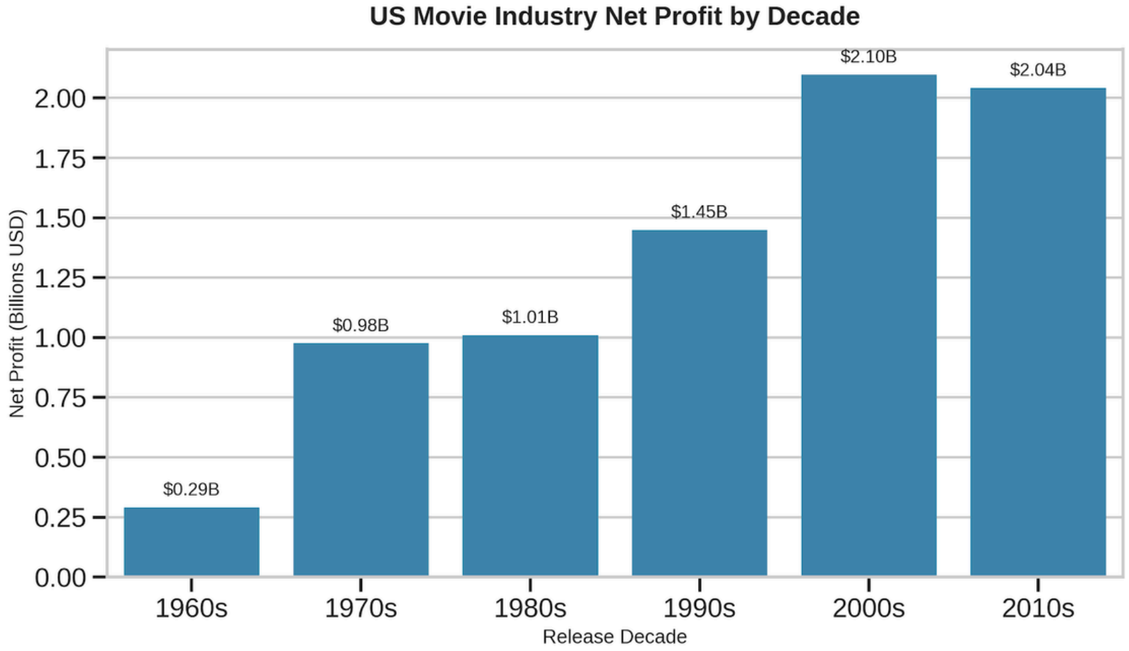
Analyzing Studio Dominance by mapping the top 10 Production Companies against their flagship Action blockbusters.

Historical & Seasonal Trends

Tracking Profitability & Seasonality

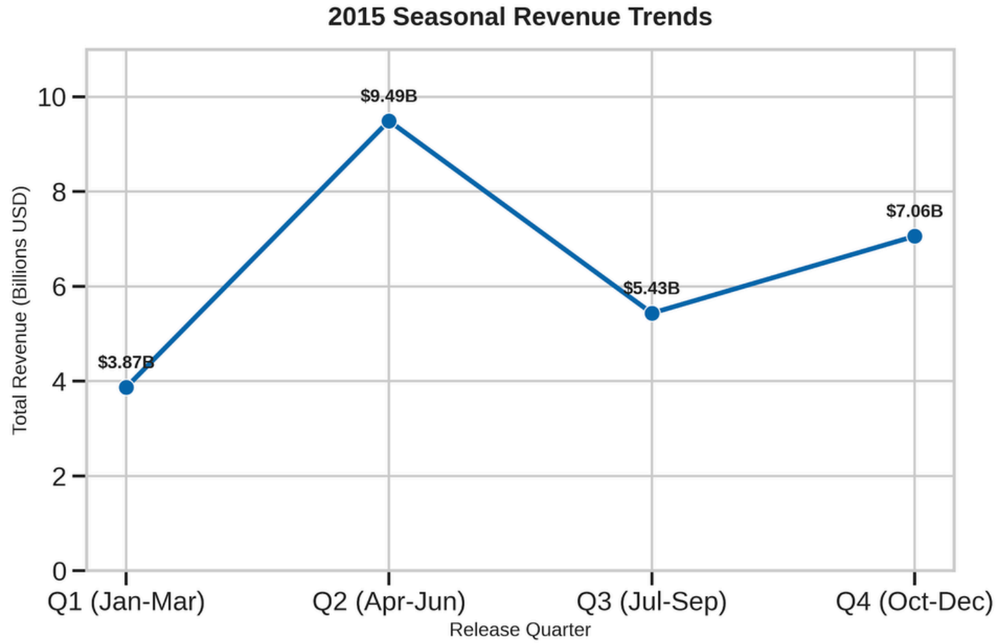
The "Golden Decade"

Using SQL Window Functions (ROW_NUMBER), the OLAP view dynamically isolates the most profitable era for each country. The US market peaked in the 2000s and 2010s, generating over $2 Billion in net profit per decade.

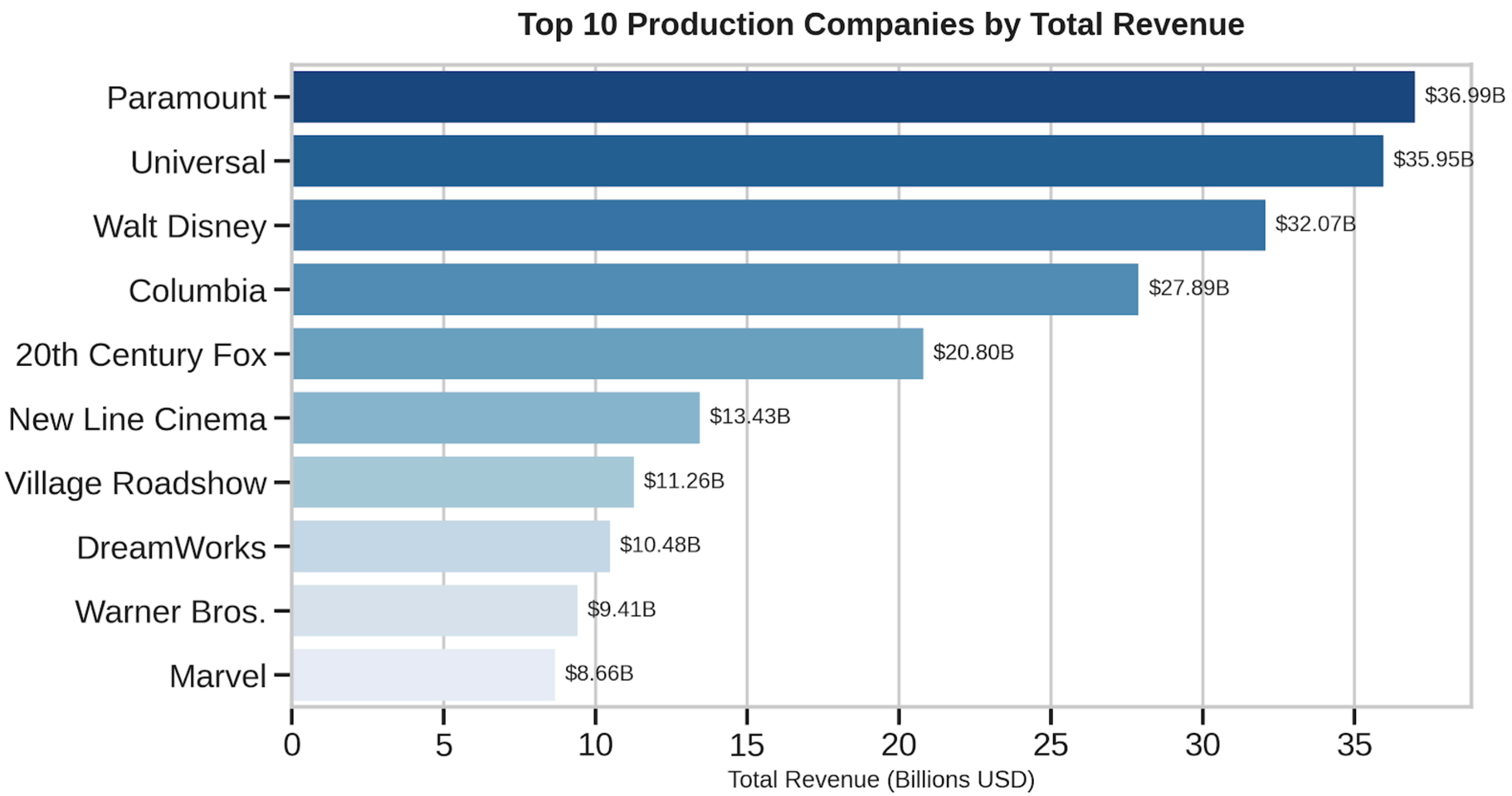


Seasonal Revenue Cycles

Aggregating monthly data into quarters (e.g., 2015) reveals strong seasonal patterns, with Q2 and Q4 historically driving the highest box office returns.



OLAP Analysis - Studio Dominance



Market Leaders & Studio Dominance

Top Tier Studios: Paramount, Universal, and Walt Disney dominate the global market, each surpassing $32 Billion in total historical revenue.

Analytical Approach: Built using aggregated LEFT JOIN operations on the Production dimension to map flagships studios against their total financial output.

Enterprise Data Quality & Automated Reconciliation

Guaranteeing Mathematical Integrity

Orphan & Negative Checks
Automated SQL assertions block negative budgets/revenues and ensure 100% referential integrity across all dimensions.

Anti Fan-out Reconciliation
A custom mathematical test verifies that OLAP aggregations do not falsely multiply revenue due to many-to-many relationships (like actors).

The Sanity Check Result
Total Revenue (Fact Table) = $X
Total Revenue (Via Dimension Joins) = $X
Status: ✅ PASS (Zero Data Loss or Duplication)

Conclusions & Future Work

Project Achievements

- Successfully integrated TMDB and IMDb without shared IDs using Virtual Staging.
- Delivered a mathematically certified Star Schema handling complex N:M relationships.

Future Work

- *API Integration:* Connect directly to the TMDB API for incremental daily updates.
- *BI Dashboarding:* Connect the PostgreSQL warehouse to Tableau or Power BI for interactive executive dashboards.